



香 港 大 學

THE UNIVERSITY OF HONG KONG

AIF-HKU Capstone Project

Name: Liu Zhaolin

Date: 01/02/2026

CONTENTS



- 1. Generation and Visualization of Mixed Images**
- 2. Measurement and Visualization of 6-Axis Force**
- 3. Current Technical Difficulties**

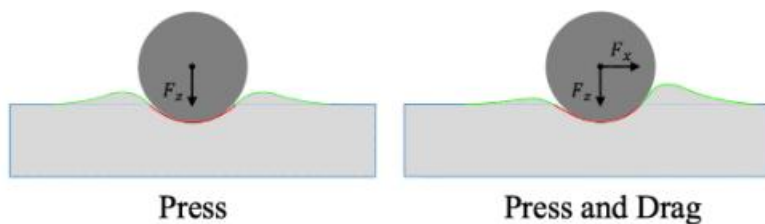
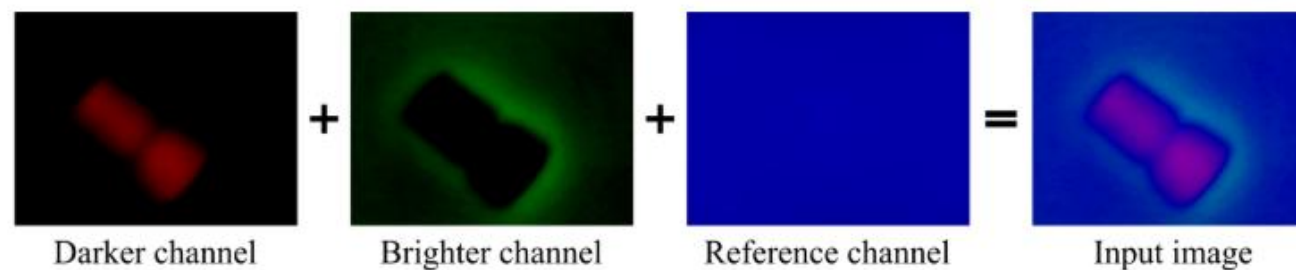


01.

Generation and Visualization of Mixed Images

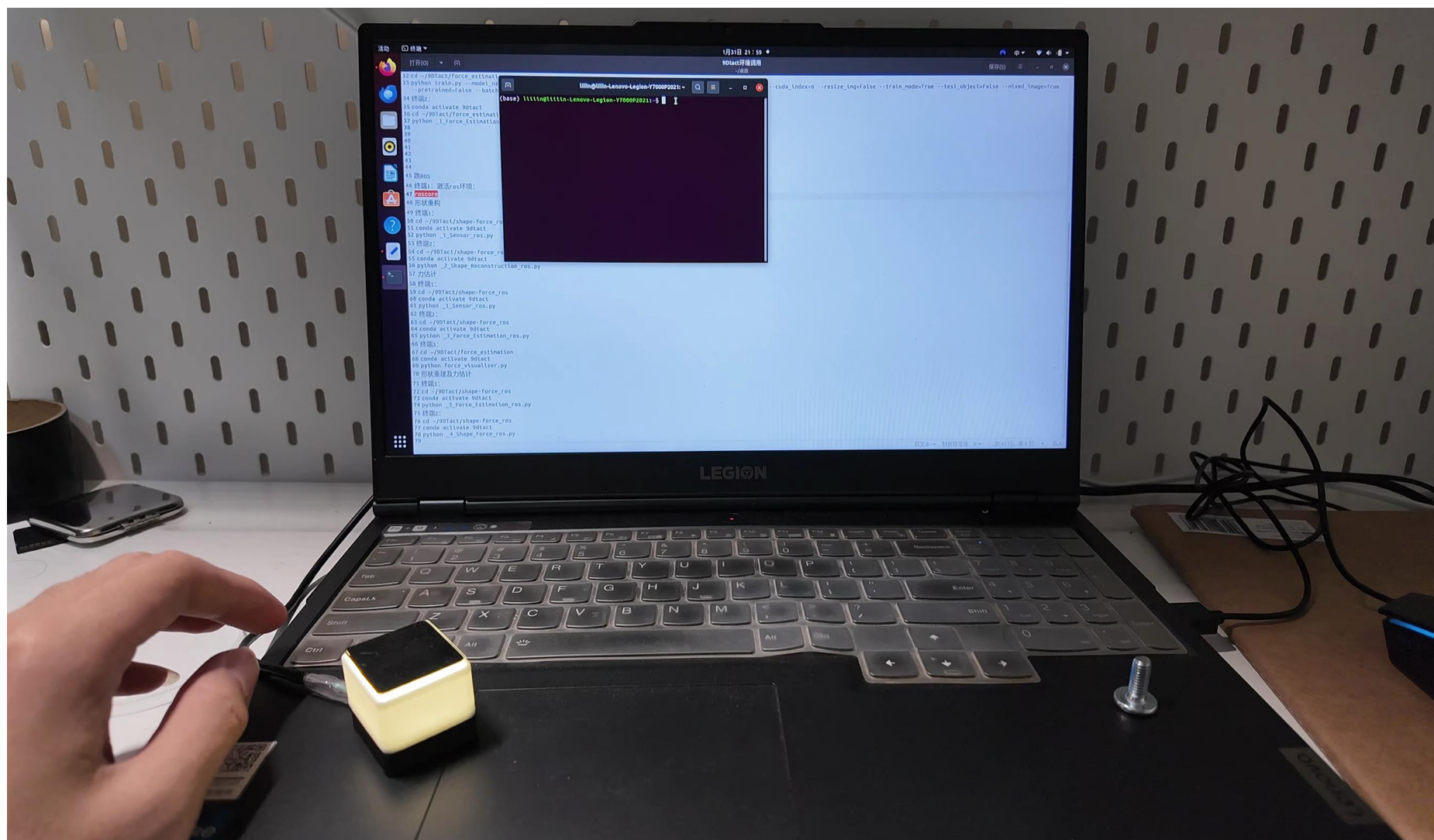
1.1 Fusion Mechanism and Physical Significance of Mixed Images

1. Darker Channel: contact geometry;
2. Brighter Channel: the dragging and twisting motions of the contact geometry;
3. Reference Channel: a sensor's reference state.

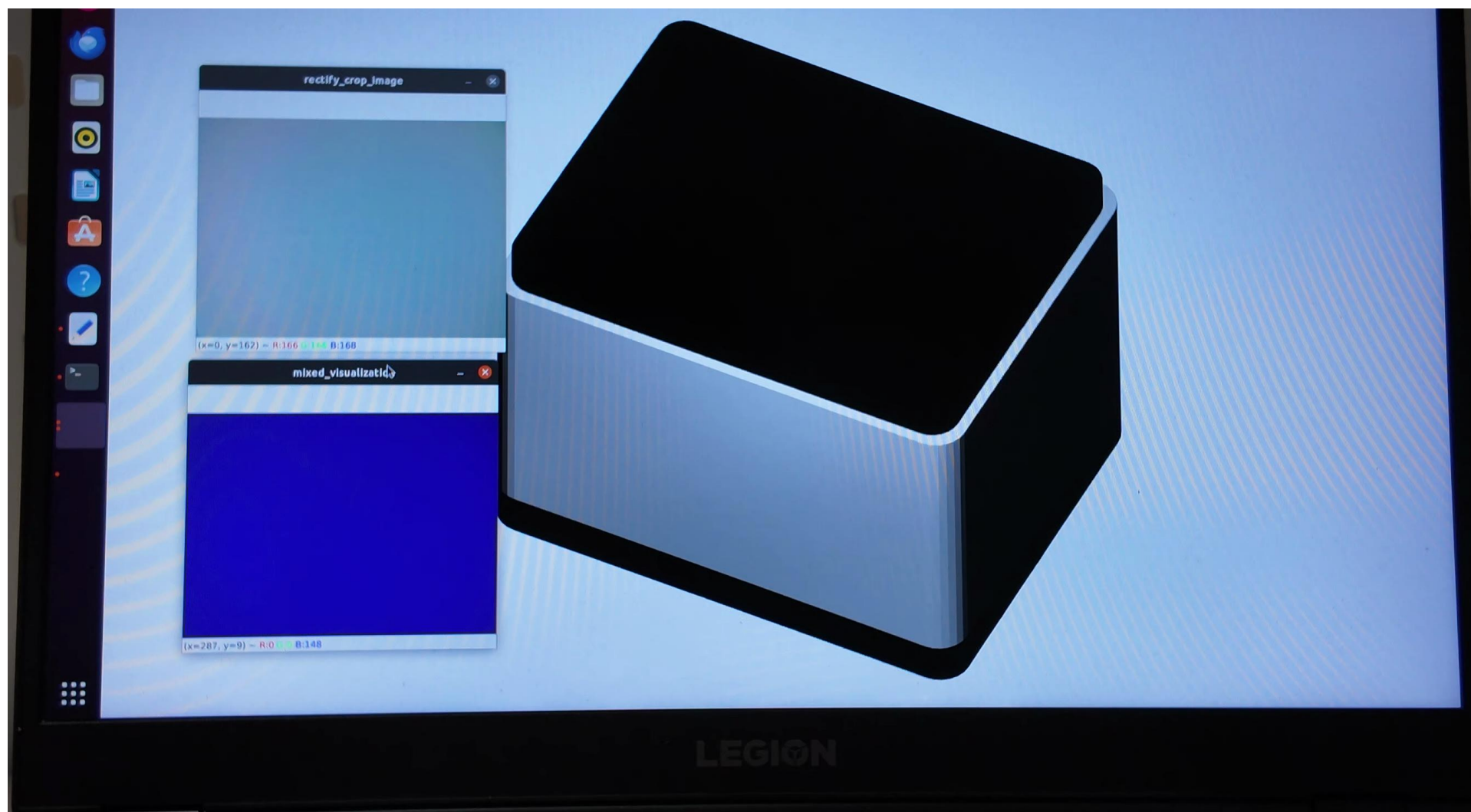


Normal deformation → Contact areas → Thinner → Darker
Shear deformation → No-contact areas → Thicker → Brighter

1.2 Demonstration of Mixed Image Generation Procedures



1.2 Demonstration of Mixed Image Generation Procedures



1.3 Code Implementation for Mixed Image Visualization

```
import numpy as np
import open3d
from open3d import *
from math import sqrt
import rospy
from geometry_msgs.msg import WrenchStamped
```

```
final_force = False
reference_frame = False
```

```
def get_rotation_matrix_from_2_vectors(v2, v1=None):
    if v1 is None:
        v1 = [0, 0, -1]
    v2 = v2 / np.linalg.norm(v2)
    v1 = v1 / np.linalg.norm(v1)
    if np.allclose(v1, v2):
        return np.eye(3)
    if np.allclose(v1, -v2):
        return -np.eye(3)
```

```
    origin = v1
    target = np.array([-1, -1, 1]) * v2
    target[0], target[1] = target[1], target[0]
    v = np.cross(origin, target)
    c = np.dot(origin, target)
    s = np.linalg.norm(v)
    kmat = np.array([[0, -v[2], v[1]], [v[2], 0, -v[0]], [-v[1], v[0], 0]])
    rotation_matrix = np.eye(3) + kmat + kmat.dot(kmat) * ((1 - c) / (s ** 2))
    return rotation_matrix
```

```
def draw_vector(x, y, z, site, cylinder_radius=0.5, cone_radius=0.7, cone_height=1):
    x_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(x), cone_height=cone_height)
```

```
def draw_vector(x, y, z, site, cylinder_radius=0.5, cone_radius=0.7, cone_height=1):
    x_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(x), cone_height=cone_height)
    y_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(y), cone_height=cone_height)
    z_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(z), cone_height=cone_height)
    xyz = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=sqrt(x ** 2 + y ** 2 + z ** 2),
        cone_height=cone_height)
```

```
    if x > 0:
        x_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [0, np.pi / 2, 0]), [0, 0, 0])
    else:
        x_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [0, -np.pi / 2, 0]), [0, 0, 0])
```

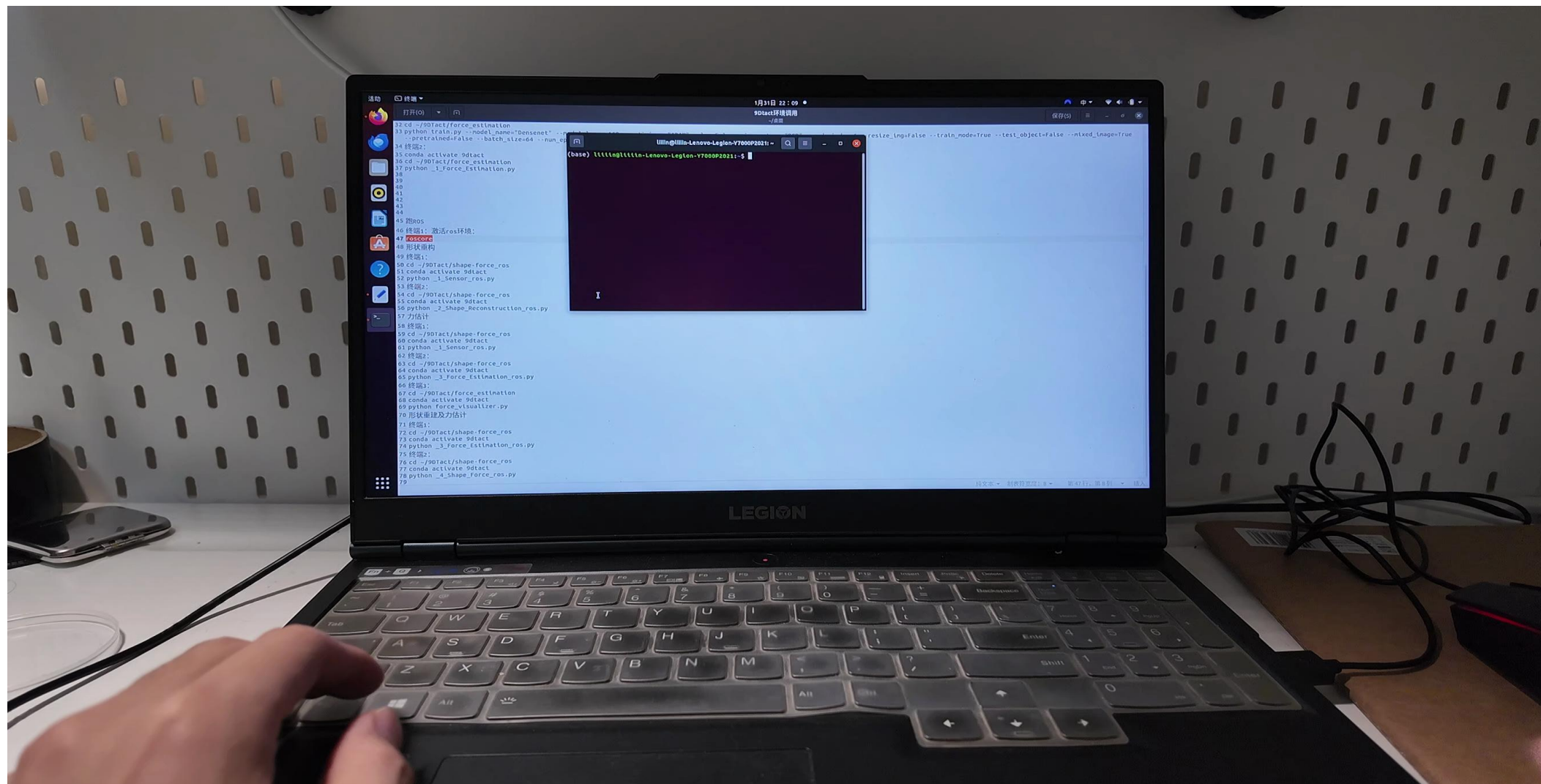
```
    if y > 0:
        y_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [np.pi, 0, 0]), [0, 0, 0])
    # if y > 0:
    #     y_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
    #         [np.pi / 0.85, 0.45, 0.45]), [0, 0, 0])
    # else:
    #     y_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
    #         [np.pi / 2, 0, 0]), [0, 0, 0])
```

```
    if z > 0:
        z_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [np.pi / 2, 0, 0]), [0, 0, 0])
    else:
        z_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [-np.pi / 2, 0, 0]), [0, 0, 0])
```

02.

Measurement and Visualization of 6-Axis Force

2.1 Demonstration of 6D Force Measurement and Visualization



2.2 Code Implementation for 6D Force Visualization

```
origin = v1
target = np.array([-1, -1, 1]) * v2
target[0], target[1] = target[1], target[0]
v = np.cross(origin, target)
c = np.dot(origin, target)
s = np.linalg.norm(v)
kmat = np.array([[0, -v[2], v[1]], [v[2], 0, -v[0]], [-v[1], v[0], 0]])
rotation_matrix = np.eye(3) + kmat + kmat.dot(kmat) * ((1 - c) / (s ** 2))
return rotation_matrix

def draw_vector(x, y, z, site, cylinder_radius=0.5, cone_radius=0.7, cone_height=1):
    x_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(x), cone_height=cone_height
    )
    y_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(y), cone_height=cone_height
    )
    z_axis = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=abs(z), cone_height=cone_height
    )
    xyz = open3d.geometry.TriangleMesh.create_arrow(
        cylinder_radius=cylinder_radius, cone_radius=cone_radius, cylinder_height=sqrt(x ** 2 + y ** 2 + z ** 2),
        cone_height=cone_height
    )

    if x > 0:
        x_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [0, np.pi / 2, 0]), [0, 0, 0])
    else:
        x_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [0, -np.pi / 2, 0]), [0, 0, 0])

    if y > 0:
        y_axis.rotate(open3d.geometry.get_rotation_matrix_from_xyz(
            [np.pi, 0, 0]), [0, 0, 0])
```

```
class Visualizer:
    def __init__(self):
        self.vis = open3d.visualization.Visualizer()
        self.vis.create_window(window_name='6D_Force_Estimation', width=2000, height=1200)

        self.force_origin = [-15, 0, 0]
        self.torque_origin = [15, 0, 0]

    if reference_frame:
        self.force_axis = draw_coordinate_frame(
            10, 10, 10, self.force_origin)
        self.force_axis_1 = draw_coordinate_frame(
            -10, -10, -10, self.force_origin)
        self.torque_axis = draw_coordinate_frame(
            10, 10, 10, self.torque_origin)
        self.torque_axis_1 = draw_coordinate_frame(
            -10, -10, -10, self.torque_origin)
        self.vis.add_geometry(self.force_axis)
        self.vis.add_geometry(self.force_axis_1)
        self.vis.add_geometry(self.torque_axis)
        self.vis.add_geometry(self.torque_axis_1)

    self.force_x, self.force_y, self.force_z, self.force_xyz = draw_vector(
        10, 10, 10, self.force_origin)
    self.torque_x, self.torque_y, self.torque_z, self.torque_xyz = draw_vector(
        10, 10, 10, self.torque_origin)

    self.last_wrench = np.ones(6)
    self.last_force = np.ones(3)
    self.last_torque = np.ones(3)

    self.vis.add_geometry(self.force_x)
    self.vis.add_geometry(self.force_y)
```

03.

Current Technical Difficulties

3.1 The Main Difficulty —— New Model Training

Why?

Since there are **unavoidable physical differences** between my sensor and the original one, the pre-trained model **doesn't fit** my hardware well.

On top of that, I plan to **optimize the structure**—like making the gel thinner and changing the camera model. This will lead to even **larger gaps** between the data.

So, I can't use the open-source model anymore. I have to train a **new model** from scratch to make sure the force measurement is accurate.

How to do next?

Requirement: High-precision Torque Sensor.

Role: Provides **Ground Truth** data to pair with tactile images.

Method: Collaborative/Synchronized Training.

Goal: Use Deep Learning to map **visual deformation features** to **physical force vectors**.

3.2 Selection and Market Research of Force/Torque Sensors

Display copy: manufacturer product images omitted.

This page originally documented reference force/torque sensor selection.

Refer to manufacturer sources for product specifications and images.

No test data or measured performance is reported on this page.



香 港 大 學

THE UNIVERSITY OF HONG KONG

Thanks!

Name: Liu Zhaolin

Date: 01/02/2026